Assignment 2

Deadline: 6th March 2026, 11:59 PM

Documentation

- General: https://grpc.io/docs/
- C++: https://grpc.io/docs/languages/cpp/basics/
- Python: https://grpc.io/docs/languages/python/
- Go: https://grpc.io/docs/languages/go/basics/
- Protobuf: https://protobuf.dev/overview/

Resources

- https://github.com/grpc-ecosystem/awesome-grpc

1 K-Nearest Neighbours (25 Points)

The k-Nearest Neighbors (k-NN) algorithm is an instance-based machine learning algorithm used for classification and regression. In k-NN, the algorithm identifies the k data points in the training dataset that are closest to a given query point and makes predictions based on these neighbors.

Implement a distributed k-NN system where:

- Multiple gRPC servers each hold a portion of the dataset.
- A client can upload data by splitting an input file across the servers.
- A client queries all servers with a data point and the value of k to find the global k-nearest neighbors.

1.1 K-NN Service API

The following API methods should be implemented:

- Client (Master Node):
  - **UploadData(UploadRequest) → UploadResponse**: The client reads a dataset file, splits it into partitions, and uploads each partition to the corresponding gRPC server. The request includes a list of `data_points`. The server stores them and returns an acknowledgment. **Each call to `UploadData` replaces any previously stored data on that server.** The client should re-partition and upload the entire dataset each time.
  - **QueryKNN(KNNRequest) → KNNResponse**: The client sends a query to each gRPC server. The request includes the `data_point` (the query point) and k (the number of nearest neighbors). Each server computes and returns the k nearest neighbors within its own subset. The client aggregates results from all servers to find the global k nearest neighbors.

1.2 Data Format

The dataset file should be a **CSV file** where each row represents a data point. Each row contains comma-separated floating-point values representing the feature dimensions. Example:

```
1.0,2.5
4.2,1.8
3.3,4.4
```

All data points in a given dataset are guaranteed to have the **same number of dimensions**. The system should support arbitrary dimensionality.

1.3 Components

1. **gRPC Servers (Workers)**

   - Each gRPC server holds a subset of the dataset, uploaded by the client.
   - When data is uploaded via `UploadData`, the server **replaces** any previously stored data with the new data.
   - When queried via `QueryKNN`, it computes the k nearest neighbors within its subset and returns them to the client.
   - Each server should accept its port number as a **command-line argument**. Example:

   ```
   python knn_server.py 50051
   python knn_server.py 50052
   python knn_server.py 50053

   ```

2. **gRPC Client (Master Node)**

   - At startup, the client takes the list of server addresses as input (e.g., `localhost:50051 localhost:50052 localhost:50053`).
   - The client provides a CLI with the following options:
     (a) **Add Data**: Reads a dataset file, splits it evenly across all servers, and uploads each partition via `UploadData`.
     (b) **Find KNN**: Takes a query data point and k as input. Sends the query to all servers, receives partial results, and aggregates them to find the global k nearest neighbors.
     (c) **End**: Exit the program.

1.4 Task Breakdown

1. **Dataset Partitioning**: The client reads the input file and splits the data points into approximately equal **contiguous chunks** across the available servers. If the number of data points is not evenly divisible by the number of servers, the remaining points should be distributed among the first few servers (each getting one extra point).
2. **Distance Calculation**: Use Euclidean distance as the metric for calculating the nearest neighbors.
3. **Result Aggregation**: The client collects the k nearest neighbors from each server, merges them, and returns the overall k nearest neighbors.
4. **Output Format**: The `QueryKNN` response should include the k nearest data points along with their **Euclidean distances** from the query point.
5. **Tie-Breaking**: If two data points have the same distance from the query point, tie-breaking is arbitrary (any consistent ordering is acceptable).

1.5 Example

Suppose the client starts with 3 servers and uploads the following 2D dataset file (`data.csv`):

```
1.0,2.0
3.0,4.0
5.0,6.0
7.0,8.0
2.0,1.0
4.0,3.0
```

Step 1: Upload Data

The client splits the 6 data points into 3 contiguous chunks (2 points each):

- Server 1 receives: $(1.0, 2.0)$, $(3.0, 4.0)$
- Server 2 receives: $(5.0, 6.0)$, $(7.0, 8.0)$
- Server 3 receives: $(2.0, 1.0)$, $(4.0, 3.0)$

Step 2: Query KNN

The client queries with point $(2.0, 3.0)$ and $k = 3$.
Each server computes the 3 nearest neighbors (or all points if it has fewer than k):

- Server 1 returns: $(1.0, 2.0)$ [dist $= \sqrt{2} \approx 1.41$], $(3.0, 4.0)$ [dist $= \sqrt{2} \approx 1.41$]
- Server 2 returns: $(5.0, 6.0)$ [dist $= \sqrt{18} \approx 4.24$], $(7.0, 8.0)$ [dist $= \sqrt{50} \approx 7.07$]
- Server 3 returns: $(2.0, 1.0)$ [dist $= \sqrt{4} = 2.0$], $(4.0, 3.0)$ [dist $= \sqrt{4} = 2.0$]

Step 3: Aggregation

The client merges all results and selects the global 3 nearest:

```
Global 3-Nearest Neighbors of (2.0, 3.0):
  (1.0, 2.0)  - distance: 1.41
  (3.0, 4.0)  - distance: 1.41
  (2.0, 1.0)  - distance: 2.00
```

1.6 Deliverables

File Requirements

- **Proto File**: Define `KNNService` and messages.
- **knn_server.py**: gRPC service implementation with data storage and distance calculation.
- **knn_client.py**: Client implementation with data upload, multi-server querying, and result aggregation.
- **README.md**: Document how to run the your program.

2 Distributed Task Queue Management (35 Points)

In this problem, you are required to build a distributed task queue system where multiple clients (Job Creators) submit tasks, and multiple workers (Task Processors) process them. Both the Job Client and the Worker Client connect to the central Task Queue Server.

2.1 Task Queue Service API

The following API methods should be implemented:

- Job Client:
  - **SubmitTask(TaskRequest) → TaskResponse**: Job Clients can submit a task by specifying the `task_type` (CPU/IO) and `payload`. The server queues the task and returns a unique `task_id`.
  - **GetTaskStatus(TaskStatusRequest) → TaskStatusResponse**: Job Clients can check the current status of their submitted task ("Queued", "Processing", "Completed", "Failed"). The Job Client should poll this method periodically and print the status to the terminal whenever it changes.
- Worker Client:
  - **RegisterWorker(WorkerConfig) → WorkerId**: Workers register themselves with their capabilities. A worker can be one of three types:
    - **CPU**: Can process only CPU-intensive tasks.
    - **IO**: Can process only I/O-intensive tasks.
    - **BOTH**: Can process both CPU and I/O tasks.

    The server returns a unique `worker_id`.
  - **RequestTask(WorkerId) → Task**: Workers request tasks from the server. If no task is available, the worker should **wait** until one is assigned and the server pushes the task to the worker when available. The server assigns the first available task from the matching FIFO queue.
  - **SendHeartbeat(Heartbeat) → Acknowledge**: Workers send periodic heartbeats to indicate they are alive. If the server misses 3 consecutive heartbeats, it marks the worker as dead. (See Section 2.4 for details.)
  - **UpdateTaskStatus(TaskStatusUpdate) → Acknowledge**: Workers update the status of a task they are processing (e.g., "Processing", "Completed", "Failed").

2.2 Payload Format

The `payload` field in the task request is a **string**:

- **CPU Task**: The payload is a string representation of an integer N (the matrix dimension). Example: `"100"` means a 100×100 matrix multiplication.
- **I/O Task**: The payload is a string representation of the sleep duration in seconds. Example: `"5"` means the worker sleeps for 5 seconds.

2.3 Task Queue and Assignment

- The server maintains **two separate FIFO queues**: one for CPU tasks and one for IO tasks.
- When a worker's stream is open and a matching task is available, the server assigns the **first task** from the queue matching the worker's capability.
- A **BOTH** worker can receive tasks from either queue. If both queues have tasks, the server assigns the task with the **earliest arrival time** across both queues.
- Workers that are currently processing a task should not receive new tasks until they have completed or failed their current task.

- Workers do not need to return the result of the computation. The task simulation is only to create realistic CPU/IO load. Upon completion, the worker simply updates the task status to `Completed` via `UpdateTaskStatus`.

2.4 Failure Detection and Reassignment

- Workers must send heartbeats every **5 seconds**.
- The server checks for heartbeats every 5 seconds. If a worker has not sent a heartbeat for **15 consecutive seconds** (i.e., 3 missed intervals), the server marks it as dead.
- When a worker failure is detected, the server must:
  1. Mark the worker's in-progress task status as **"Failed"** and close the worker's stream.
  2. The Job Client will see the `Failed` status when it polls via `GetTaskStatus`.
  3. Re-queue the task at the **front** of the corresponding queue, setting its status back to **"Queued"** for immediate reassignment to another available worker.

2.5 Task Simulation

- **CPU Task**: The worker performs a matrix multiplication of size $N \times N$ (where N is determined by the payload).
- **I/O Task**: The worker sleeps for a duration determined by the payload, simulating a network or database call.

Note: The server address should be accepted as a **command-line argument** by the server, workers, and Job Clients.

2.6 Sample Scenario

The following illustrates a typical interaction:

```
# Terminal 1: Start server
$ python server.py localhost:50051
[Server] Listening on localhost:50051

# Terminal 2: Register a CPU worker
$ python worker.py localhost:50051 CPU
[Worker] Registered with ID: W1 (type: CPU)
[Worker] Waiting for tasks...

# Terminal 3: Register an IO worker
$ python worker.py localhost:50051 IO
[Worker] Registered with ID: W2 (type: IO)
[Worker] Waiting for tasks...

# Terminal 4: Submit tasks via Job Client
$ python client.py localhost:50051
> submit CPU 100
[Client] Task submitted. ID: T1 (CPU, payload=100)
> submit IO 5
[Client] Task submitted. ID: T2 (IO, payload=5)
> status T1
[Client] Task T1: Processing (Worker W1)
> status T2
[Client] Task T2: Processing (Worker W2)
> status T1
[Client] Task T1: Completed
```

Meanwhile, in Terminal 2 (CPU Worker):

```
[Worker W1] Received task T1 (CPU, payload=100)
[Worker W1] Performing 100x100 matrix multiplication...
[Worker W1] Task T1 completed. Waiting for next task...
```

If Worker W1 crashes (stopped sending heartbeats), the server detects failure after 15 seconds, marks the task as `Failed`, re-queues it, and assigns it to another available CPU worker.

2.7 Deliverables

File Requirements

- **Proto File**: Define `TaskQueueService` and messages.
- **server.py**: Implements queue logic, failure detection, and RPCs.
- **worker.py**: Implements worker logic, heartbeats, and task processing.
- **client.py**: Implements task submission and status polling.
- **README.md**: Documentation on how to run the system.